

AUTONOMOUS SEMANTIC NAVIGATION ROBOT

Project Source of Truth - Version 1.1

Approved baseline: 21 September 2026 | Fall 2026 execution window: 31 August - 13 December 2026

Purpose. This document prevents scope drift. It defines what the project is, what V1/V2/V3 must accomplish, the deadlines, required tools/materials, budget ceilings, ASU acquisition strategy, decision gates, and what is explicitly out of scope. Repository structure and detailed implementation architecture will be defined in the next project document.

1. Locked project objective

Build a professionally engineered indoor autonomous mobile robot platform that demonstrates modern C++, ROS 2, mapping/localization, sensor fusion, autonomous navigation, 3D perception, sim-to-real deployment, testing, and quantitative validation.

Career objective: create credible, interview-defensible evidence for robotics software, autonomy, perception, integration/test, and related internship roles. The project is intentionally designed around the current gaps identified in the Summer 2027 internship system: advanced C++, ROS/ROS2, SLAM/localization, production robotics software, real-time/control implementation, 3D vision, sensor fusion, and motion planning.

2. Change-control rules

Status	Meaning
LOCKED	Do not change without creating a new document version and recording why.
OPEN	Decision intentionally deferred until more information is available.
STRETCH	Valuable only after the mandatory version scope works reliably.
OUT OF SCOPE	Not a Fall 2026 requirement; do not let it delay the core system.

Locked decisions. Three-stage development: V1 simulation -> V2 physical autonomy -> V3 semantic perception and rigorous validation. No hardware purchase before the hardware decision gate. Personal spending ceiling for Fall 2026: approximately \$1,500, with a preferred outcome of \$0-\$600 by using ASU resources.

3. Fall 2026 master timeline

Phase	Dates	Deadline	What must be true at exit
Phase 0 - Preparation	Aug 31 - Sep 27	Sep 27	C++ foundation complete; ROS-ready Linux environment; project requirements and hardware-access investigation underway.
V1 - Simulation	Sep 28 - Oct 25	Oct 25	Simulated robot maps, localizes and autonomously reaches goals using ROS 2 + Nav2 without physical hardware.
V2 - Physical autonomy	Oct 26 - Nov 22	Target Nov 15; hard Nov 22	Same autonomy stack works on a real mobile robot with real sensors; controlled navigation test campaign completed.

Phase	Dates	Deadline	What must be true at exit
V3 - Semantic + robust autonomy	Nov 16 - Dec 13	Dec 13	Robot uses RGB-D perception for semantic/3D targets and has a repeatable validation pipeline with measured results.

Note: technical learning and project versions overlap intentionally. For example, visual-perception coursework can begin while V2 hardware integration is still being stabilized.

4. Phase 0 - Preparation (Aug 31 - Sep 27)

Core rule. Do not buy the robot now. Phase 0 is learning, design, environment setup, and hardware-access discovery.

4.1 Mandatory scope

- Complete the planned C++ fundamentals sequence: syntax, control flow, functions, STL/basic data structures, references/pointers, classes/OOP, file I/O, and CMake basics.
- Set up a clean Linux development environment and Git/GitHub workflow.
- Create the project requirements list and architecture diagrams; no claim that hardware or ROS functionality exists until implemented.
- Research and confirm ASU access to a TurtleBot-class robot, LiDAR, RGB-D/depth camera, Jetson/Raspberry Pi, electronics benches, fabrication tools, and safe test space.
- Select the V2 hardware path no later than the hardware decision gate.

4.2 Tools

Tool	Purpose	Cost
Ubuntu 24.04	Primary development OS; native Ubuntu 24.04 dual-boot environment.	\$0
C++ / GCC	Core robotics software language/toolchain.	\$0
CMake	Build configuration for C++ and later ROS 2 packages.	\$0
Git + GitHub	Version control, issues, milestones and public portfolio evidence.	\$0
Editor/IDE	VS Code or equivalent.	\$0

4.3 Exit criteria

- Can build and debug a multi-file C++ program with CMake.
- Can explain the high-level robot pipeline: sensors -> state estimate -> map/localization -> planner/controller -> motor commands.
- ASU hardware request has been sent and at least one concrete access path is being pursued.
- No unnecessary hardware has been purchased.

5. V1 - Simulated Autonomous Robot (Sep 28 - Oct 25)

V1 mission. Create a differential-drive indoor robot in simulation that can build/use a map, estimate its pose, plan a collision-free path, and autonomously drive to commanded goals. V1 proves the software architecture before money is committed to hardware.

5.1 Mandatory scope

Subsystem	V1 requirement
ROS 2 foundation	Own workspace/packages; C++ publishers/subscribers/services/actions as needed; launch/config files.
Robot model	Differential-drive robot model with correct frames, dimensions, wheel geometry and sensor placement.

Subsystem	V1 requirement
Simulation	Isaac Sim world(s) with realistic walls/obstacles and simulated wheel odometry + 2D LiDAR.
TF / state	Correct TF tree and robot_state_publisher; inspect transforms in RViz.
Mapping	Use SLAM Toolbox to create and save an indoor map.
Localization	Reload a saved map and localize the robot.
Navigation	Nav2 goal execution, global/local planning, collision avoidance and recovery behavior.
Own C++ work	At least one non-trivial C++ ROS 2 component owned by the project (mission/health/telemetry or equivalent); not just launching a stock demo.
Logging	Record representative runs with rosbag and preserve configuration/version information.

5.2 V1 acceptance criteria

- Create a map of at least one custom simulated indoor world and save it.
- Restart the system using the saved map and obtain stable localization.
- Execute at least 10 predefined navigation goals in simulation.
- Complete the demonstration without intentional manual steering.
- Document at least two observed failure modes and the configuration/debugging changes used to address them.
- Produce a reproducible launch procedure from a clean terminal.

5.3 V1 budget and materials

Item	Required?	Budget
Computer capable of Linux development	Yes - already available	\$0 incremental
ROS 2 Jazzy + RViz + Nav2 + SLAM Toolbox	Yes	\$0
Isaac Sim simulator	Yes	\$0
Physical robot/sensors	No	\$0
V1 personal budget		\$0

5.4 V1 software baseline

Baseline stack: native Ubuntu 24.04 + ROS 2 Jazzy + NVIDIA Isaac Sim + RViz + Nav2 + SLAM Toolbox + tf2 + rosbag + CMake/colcon. ROS 2 Jazzy officially supports Ubuntu 24.04. Isaac Sim is the locked V1 simulator.

6. V2 - Physical Autonomous Robot (Oct 26 - Nov 22)

V2 mission. Transfer the autonomy stack from simulation to a real indoor mobile robot, integrate real odometry/IMU/LiDAR, fuse state estimates, calibrate the platform, and demonstrate reliable autonomous navigation.

6.1 Preferred hardware path

Priority	Path	Decision
1	Borrow/use an ASU TurtleBot 4 / TurtleBot 4 Lite or equivalent platform.	Preferred - minimizes cost and integration risk.
2	Borrow individual sensors/compute and build only the missing base/electronics.	Strong fallback.
3	Self-fund a custom differential-drive robot.	Fallback if ASU platform access fails.
4	Buy a complete TurtleBot 4 Lite.	Last resort; requires current vendor quote and strong justification.

6.2 Mandatory V2 capabilities

Subsystem	V2 requirement
Physical motion	Reliable differential-drive motion on real floor surfaces with emergency/stop procedure.
Odometry	Use wheel encoder odometry; understand wheel radius/separation calibration and drift.
IMU	Integrate IMU orientation/angular-rate data.
Sensor fusion	Fuse odometry + IMU with robot_localization EKF (or equivalent) and compare against unfused behavior.
LiDAR	Integrate 2D LiDAR scan stream with correct frame/calibration.
SLAM/localization	Create a real indoor map; save it; later localize against it.
Navigation	Use Nav2 for autonomous point-to-point navigation and obstacle avoidance.
Own C++ work	By V2, implement/own at least one meaningful C++ robotics component beyond configuration. Exact component will be locked in the implementation-structure document.
Validation	Run a fixed test set repeatedly and log success/failure, intervention, collisions, path time and other meaningful metrics.

6.3 V2 acceptance criteria

- Map at least one real indoor test area and reuse the saved map on a separate run.
- Perform at least 20 predefined autonomous navigation trials.
- Target: $\geq 90\%$ goal completion without manual intervention in the controlled test set; zero intentional collisions.
- Record and compare localization/navigation behavior with and without the selected sensor-fusion configuration.
- Demonstrate at least one recovery/failure case (for example temporary blocked path) and document behavior.
- Hardware, software version, calibration values, wiring and launch instructions are documented well enough to reproduce the setup.

6.4 V2 material list - custom-build fallback

Material	Purpose	Planning range
Differential-drive chassis + 2 encoder motors + wheels/caster	Physical base and measured wheel motion.	\$120-\$220
Motor driver / low-level controller	Drive motors from commands.	\$25-\$60
Microcontroller (if required)	Encoder/low-level motor interface.	\$15-\$35
2D LiDAR (RPLIDAR A1 class)	360-degree range sensing for mapping/navigation.	~\$99
IMU	Angular-rate/orientation input if platform does not already include one.	\$15-\$40
Battery + charger + regulators + protection	Portable, stable power.	\$70-\$120
Onboard compute	Raspberry Pi / reused compatible compute; exact choice OPEN.	\$0-\$120
Cables, connectors, fasteners, mounts, printed parts	Integration and mechanical fit.	\$50-\$100
Contingency	Shipping, replacement connectors, small integration surprises.	\$50-\$100
CUSTOM V2 PLANNING RANGE		\$450-\$700

6.5 V2 budget scenarios

Scenario	Personal spending target	Notes
ASU complete platform	\$0-\$150	Preferred. Mostly adapters, mounts, consumables or personal spares.
Hybrid ASU + own missing parts	\$150-\$450	Buy only the components ASU cannot provide.
Custom mobile base	\$450-\$700	Baseline self-funded V2 fallback.

Scenario	Personal spending target	Notes
Purchase complete TurtleBot 4 Lite	~\$1,200+; verify quote	Not baseline. Public indicative pricing exists, but current official vendor price must be checked before purchase.

7. V3 - Semantic and Robust Autonomous Robot (Nov 16 - Dec 13)

V3 mission. Add RGB-D perception so the robot can reason about objects in 3D and convert semantic detections into navigation goals, while turning the entire system into a measured, repeatable engineering test platform.

7.1 Mandatory V3 scope

Subsystem	V3 requirement
RGB-D input	Use a depth camera already on the platform or borrowed/purchased equivalent.
Object perception	Run a practical object detector/segmenter; pretrained models are allowed. Do not spend Fall 2026 training a new foundation model.
3D localization	Use depth + camera calibration to estimate object position; transform detections into robot/map frames with tf2.
Semantic navigation	Select a detected target/object and generate a safe navigation goal near it.
Robust software	Health/status logging, sensible failure handling, configuration management and reproducible startup.
Validation harness	Repeatable scenario/test definitions, rosbag capture, metrics and result summaries.
Sim-to-real comparison	Run at least one comparable scenario in simulation and physical hardware and discuss differences/failure causes.

7.2 V3 acceptance criteria

- Detect at least one agreed target class in an indoor scene and estimate its 3D position.
- Transform the detected position to the map/world frame and create a valid navigation target.
- Robot autonomously approaches the target from multiple starting positions.
- Run a documented semantic-navigation test set with quantitative results.
- Produce a final demo video, technical README/report, architecture diagram and results table.
- Document at least three system limitations/failure modes honestly.

7.3 V3 hardware choices

Item	Status	Current planning cost	Rule
Depth camera already on borrowed robot (e.g., OAK-D-Lite on TurtleBot 4 Lite)	Preferred	\$0 personal	Use before buying another camera.
Borrowed ASU RGB-D camera	Preferred	\$0 personal	Use if compatible and available.
RealSense D435i class camera	Fallback	~\$334	Purchase only if V3 requires a personal RGB-D sensor.
Jetson Orin Nano Super Developer Kit	OPTIONAL / STRETCH	\$249 MSRP; retail can vary	Buy only if onboard GPU/edge deployment creates real technical value.

7.4 V3 budget scenarios

Scenario	V3 incremental spend	Approx. Fall project total
ASU platform already has depth camera; laptop/RPi compute	\$0-\$100	\$0-\$250
ASU platform + personal Jetson if justified	\$249-\$400 + small integration items	~\$300-\$600

Scenario	V3 incremental spend	Approx. Fall project total
Custom V2 + purchased depth camera	~\$334 + integration	~\$800-\$1,100
Custom V2 + depth camera + Jetson	~\$583-\$734 + integration	~\$1,100-\$1,500

Budget ceiling. Fall 2026 personal spending should not exceed approximately \$1,500 without a new explicit decision. The target is substantially lower because ASU equipment access is expected to be investigated first.

8. Hardware acquisition and ASU strategy

8.1 What ASU access is confirmed publicly

- Polytechnic Innovation Hub (Tech Center 199) lists 3D printers, laser cutters, ShopBot, vacuum forming, injection molding and hand tools. Training is required for relevant equipment.
- Peralta Engineering Studio lists electronics/manufacturing resources; Peralta 109 includes PCB mills, work tables, bandsaw, drill press and hand tools, and the lab states semester project lockers can be requested.
- ASU Innovation Showcase projects from RAS 598 have used TurtleBot 4 / TurtleBot 4 Lite platforms with ROS 2, LiDAR, IMU, depth cameras, SLAM and sensor fusion.

Important limitation. Public evidence confirms ASU owns/uses relevant robotics equipment, but it does NOT confirm that this student can automatically check out a TurtleBot or sensors. Professor/lab approval and actual availability remain OPEN decisions.

8.2 First hardware-access request

The professor/lab conversation should specifically ask about:

- TurtleBot 4, TurtleBot 4 Lite, or another ROS 2-capable differential-drive mobile robot.
- 2D LiDAR (RPLIDAR-class), RGB-D/depth cameras, IMUs and spare sensors.
- Raspberry Pi / Jetson or other onboard compute suitable for ROS 2.
- A safe indoor test area and expected access hours/reservation rules.
- Permission to use lab equipment for an independent portfolio/research project.
- Faculty mentorship or a way to align the project with RAS coursework/research.
- Storage for the robot/project between sessions.

8.3 Acquisition decision gates

Date	Gate	Action
Sep 4	Initial ASU contact made	Professor/lab request sent; identify at least one backup contact/resource.
Sep 18	Access status review	Know whether a complete robot is likely, individual components may be available, or no hardware support is expected.
Sep 27	Hardware architecture lock	Choose ASU complete platform, hybrid, or custom-build path.
Oct 12	Procurement readiness	If custom path is needed, final BOM is approved; no speculative components.
Oct 19	Purchase deadline	Order missing V2 parts early enough for Oct 26 physical-integration start.
Nov 9	V3 sensor gate	Decide whether a separate depth camera or Jetson is genuinely necessary.

8.4 Funding

ASU's published MORE program information historically lists a \$1,500 student stipend and eligibility to apply for up to \$400 in research supplies. However, the currently public page still references Fall 2025 / Spring 2026 timing. Therefore MORE is a potential future funding route only and is NOT assumed in this Fall 2026 budget until ASU confirms a current cycle.

9. Scope boundaries - what NOT to build in Fall 2026

Item	Status	Reason
Humanoid or legged robot	OUT OF SCOPE	Massive mechanical/control scope; weak return per unit of time/cost for this semester.
Autonomous road vehicle / RC self-driving car as primary platform	OUT OF SCOPE	Adds steering/outdoor/high-speed problems that do not improve the core internship evidence enough.
Robot arm / mobile manipulation	STRETCH for Spring 2027	Valuable after a reliable mobile base exists; not required for Fall success.
Train a large perception/foundation model from scratch	OUT OF SCOPE	Consumes compute/data/time; integration and robotics engineering are the priority.
VLM/VLA natural-language robot control	STRETCH for Spring 2027	Potential extension only after V3 is reliable.
Multi-robot autonomy	STRETCH	Useful but not necessary to close current skill gaps.
Write SLAM from scratch	OUT OF SCOPE	Use a production-quality ROS package and focus on integration, understanding, calibration and evaluation.
Buy expensive hardware because it looks impressive	OUT OF SCOPE	Hardware is purchased only when it unlocks a defined requirement.

10. What must be personally demonstrated by the end

- Modern C++ used in real ROS 2 packages/components, not only course exercises.
- Ability to explain ROS 2 nodes, topics, services/actions, TF, launch/configuration, build tooling and debugging.
- Mapping/localization and why SLAM differs from localization on a known map.
- Wheel odometry, IMU data, drift and the purpose of sensor fusion/EKF.
- Nav2 planning/control architecture and at least one meaningful personally owned C++ robotics component.
- Real sensor integration and calibration issues.
- RGB-D/3D perception and coordinate-frame transformations.
- Simulation-to-real differences.
- Systematic robotics validation: test cases, logs, metrics, repeatability and failure analysis.

11. Project success definition for December 13

Minimum successful Fall outcome. A reproducible ROS 2/C++ mobile-robot system works in simulation and on real hardware, builds/uses maps, localizes, fuses real sensors, navigates autonomously, recognizes and spatially localizes at least one semantic object class, navigates toward it, and is supported by quantitative tests, logs, documentation and a demo.

The project is NOT considered complete merely because:

- a stock TurtleBot/Nav2 demo launches;
- the robot reaches one goal once;
- a camera model detects an object on a laptop;
- a tutorial was followed without understanding/configuration ownership;
- there are no repeatable tests or measured results.

12. Risks and fallback rules

Risk	Fallback
ASU robot cannot be borrowed	Use custom-build V2 plan; keep V1 progressing so hardware delay does not stop software development.

Risk	Fallback
Hardware shipment delay	Continue simulation/validation; use borrowed components if available; do not lower software quality to rush assembly.
Isaac Sim/GPU issues	Reduce Isaac Sim rendering/physics load or use a lab workstation when needed; keep native Ubuntu 24.04 for ROS 2/C++.
Fall coursework becomes heavy	Preserve the minimum V1/V2 core; defer Jetson, advanced semantic features and stretch work rather than sacrificing academics.
V3 is too large	Minimum V3 is one reliable semantic target pipeline + quantitative validation. Defer VLM/VLA, manipulation and advanced 3D perception.
Borrowed hardware access is limited	Simulation remains the daily development platform; schedule physical test blocks and use rosbag data offline.

13. Relationship to the weekly learning plan

Learning block	Project use
C++ (Aug 31-Sep 27)	Prepares code quality, OOP, build/debugging and later ROS 2 C++ components.
ROS 2 (Sep 28-Oct 11)	Directly creates the V1 package/communication/TF/launch foundation.
Nav2 + SLAM (Oct 12-Oct 25)	Completes V1 autonomy and prepares V2 deployment.
State estimation (Oct 26-Nov 15)	Directly supports V2 odometry/IMU fusion and localization evaluation.
Visual perception (Nov 16-Dec 6)	Directly supports V3 RGB-D perception and semantic navigation.
Dec 7-Dec 13 buffer	Integration, final validation, documentation and catch-up; no new major feature unless core is already stable.

14. Research basis and verified sources

Source-of-truth rule: prices are planning estimates, not purchase authorizations. Exact retailer availability, tax, shipping and ASU access must be rechecked at each purchase/decision gate.

- ROS 2 Jazzy Ubuntu support - Official docs state Ubuntu Noble 24.04 is supported.
<https://docs.ros.org/en/jazzy/Installation/Alternatives/Ubuntu-Install-Binary.html>
- NVIDIA Isaac Sim - Official documentation for robot simulation and ROS 2 integration on Ubuntu Linux.
<https://docs.isaacsim.omniverse.nvidia.com/latest/index.html>
- TurtleBot 4 Lite hardware - Official manual: Create 3 base, Raspberry Pi 4B, RPLIDAR A1M8, OAK-D-Lite camera.
<https://turtlebot.github.io/turtlebot4-user-manual/overview/features.html>
- ASU Innovation Hub equipment - Public ASU list of fabrication equipment and training/access rules.
<https://poly.engineering.asu.edu/innovation-hub-equipment/>
- ASU Peralta Engineering Studio - Public ASU description of manufacturing/electronics facilities and semester project storage.
<https://labs.engineering.asu.edu/peraltastudios/>
- ASU TurtleBot 4 Lite student example - RAS project using ROS2, LiDAR, IMU, depth camera, sensor fusion and SLAM.
<https://innovationshowcase.engineering.asu.edu/project/team-2-2/>
- ASU TurtleBot 4 student example - RAS project using TurtleBot 4, LiDAR, IMU, camera and ROS2 autonomous mapping/navigation.
<https://innovationshowcase.engineering.asu.edu/project/team-5-2/>
- RealSense D435i pricing/spec - Current public price observed: \$334; D435i includes an IMU.
<https://www.realsenseai.com/stereo-depth-cameras/>
- RPLIDAR A1 price reference - Current public price observed: \$99.
<https://www.dfrobot.com/product-1125.html>
- Jetson Orin Nano Super - NVIDIA developer page lists \$249; some retailer/marketplace listings differ, so budget \$249-\$400 if later justified.
<https://developer.nvidia.com/embedded/buy-jetson>
- ASU MORE program - Public page lists \$1,500 stipend and up to \$400 research supplies for funded semesters, but published application timing is stale for the current project.
<https://students.engineering.asu.edu/graduate/research/more/>

15. Open decisions for the next document

Next step: implementation structure. The next project document will lock the system architecture and repository structure: packages/nodes, interfaces/topics, what code is custom vs third-party, testing architecture, simulation worlds, metrics, GitHub folders/issues/milestones, and the exact weekly implementation sequence.

- Exact V2 physical platform after professor/ASU response.
- Exact custom C++ component(s) that will carry the strongest technical ownership.
- Exact sensor-fusion configuration and ground-truth/accuracy evaluation method.
- Exact V3 object classes and perception model.
- Whether Jetson/Isaac ROS becomes a justified stretch goal.